Universidad del Valle de Guatemala

Facultad de Ingeniería

Computación Paralela y Distribuida

Proyecto 2

Búsqueda por fuerza bruta de llaves DES con Open MPI y OpenMP

Docente: Ing. Juan Luis García

Integrantes:
- José Mérida
- Javier España
- Angel Esquit

Semestre 2, 2026
Guatemala

Índice

1. Introducción **2**

2. Parte A — Antecedentes **2**

2.1. Investigación sobre DES 2

2.2. Diagrama de flujo del algoritmo DES 4

2.3. Resultados de la versión secuencial 5

2.4. Evidencia de correcto funcionamiento del wrapper DES 5

2.5. Explicación de rutinas clave 6

2.6. Primitivas MPI utilizadas 7

3. Parte B — Análisis y acercamientos paralelos **8**

3.1. Cifrado y descifrado de un texto sencillo 8

3.2. Pruebas con `-np 4` sobre el texto de referencia 9

3.3. Acercamientos alternativos 10

3.3.1. Acercamiento 1 — Master-Worker dinámico 10

3.3.2. Acercamiento 2 — Intercalado / Striding 11

3.3.3. Acercamiento 3 — Partición contigua con shuffle pseudo-aleatorio 11

3.3.4. Acercamiento 4 — Híbrido MPI + OpenMP 12

3.4. Comparativa de tiempos (fácil / media / difícil) 13

4. Discusión, conclusiones y recomendaciones **16**

5. Anexo 1 — Catálogo de funciones y librerías **17**

6. Anexo 2 — Bitácora de pruebas y speedups **19**

7. Bibliografía **20**

1 Introducción

El cifrado DES (*Data Encryption Standard*) fue el estándar de cifrado simétrico de bloque dominante desde su adopción por el NIST en 1977 hasta finales de la década de 1990 (NIST, 1999). Fue formalmente reemplazado por AES porque su espacio de llaves de 56 bits efectivos se volvió computacionalmente inseguro frente al poder de cómputo disponible (Stallings, 2017, cap. 3). Esa debilidad es precisamente lo que hace de DES un caso de estudio clásico para la computación paralela. Su espacio de búsqueda de 2^{56} llaves es demasiado grande para ejecución manual, pero abarcable con cómputo paralelo moderno. Un ataque de fuerza bruta que resulta impráctico en una sola máquina puede volverse tratable repartiendo el espacio de llaves entre múltiples procesos y, eventualmente, múltiples máquinas.

Este proyecto implementa y analiza dicho ataque de fuerza bruta sobre DES. Parte de una versión secuencial de referencia y evoluciona hacia distintas estrategias de paralelización con **Open MPI**: una versión *naive* que particiona el espacio de llaves en bloques contiguos iguales, y tres acercamientos alternativos pensados para corregir las limitaciones observadas en la versión naive (*master-worker* dinámico, recorrido intercalado o *striding*, y partición con *shuffle* pseudo-aleatorio). Además se implementó una versión híbrida que combina paso de mensajes (MPI, memoria distribuida) con memoria compartida (**OpenMP**) dentro de cada proceso. Esto aprovecha los núcleos disponibles en cada nodo sin necesidad de lanzar un proceso MPI por núcleo.

El análisis responde tres preguntas: cómo cambia el tiempo según la posición de la llave, qué speedup y eficiencia se obtienen, y qué costos tiene cada estrategia. Las Partes A y B siguen la estructura de la rúbrica y el documento cierra con conclusiones, anexos y bibliografía.

2 Parte A — Antecedentes

2.1 Investigación sobre DES

DES opera sobre bloques de 64 bits de texto plano utilizando una llave de 64 bits (56 bits efectivos más 8 bits de paridad, uno por cada byte). El algoritmo se estructura en las siguientes etapas. Están descritas en el estándar oficial FIPS PUB 46-3 (NIST, 1999) y resumidas por Stallings (2017, cap. 3).

1. **Permutación inicial (IP):** reordena los 64 bits del bloque de entrada según una tabla fija, sin aportar seguridad criptográfica directa pero facilitando la carga en hardware de la época.
2. **16 rondas de Feistel:** en cada ronda, el bloque se divide en una mitad izquierda L y una mitad derecha R de 32 bits cada una. La función de ronda F se aplica sobre R junto con la subllave de esa ronda, y el resultado se combina mediante XOR con L. Al finalizar la ronda, L y R se intercambian.
3. **Función F:** expande R de 32 a 48 bits mediante la tabla de expansión E, realiza XOR con la subllave de 48 bits de la ronda, pasa el resultado por 8 *S-boxes* (sustitución no lineal, cada una mapea 6 bits de entrada a 4 bits de salida) y aplica una permutación final P.
4. **Generación de subllaves (*key schedule*):** la llave original de 64 bits pasa por la permutación PC-1 (que descarta los 8 bits de paridad, dejando 56 bits efectivos), rotaciones de bits por ronda, y la permutación PC-2, generando las 16 subllaves de 48 bits usadas en cada ronda.

5. **Permutación final (IP^{-1}):** inversa de la permutación inicial, produce el bloque de texto cifrado de 64 bits.

El descifrado reutiliza exactamente el mismo algoritmo, aplicando las 16 subllaves en orden inverso. Esta simetría es la razón por la que, en este proyecto, `des_decrypt` y `des_encrypt` (Sección 5) comparten casi todo su código.

La vulnerabilidad estructural de DES frente a fuerza bruta no proviene de un defecto en las S-boxes ni en el esquema de Feistel en sí. Ambos se han mantenido razonablemente resistentes a criptoanálisis diferencial y lineal. El problema real es el tamaño de su espacio de llaves efectivo, $2^{56} \approx 7{,}2 \times 10^{16}$ combinaciones. Ese número era impracticable de recorrer secuencialmente en el hardware de los años 70. Sin embargo, el proyecto EFF DES Cracker demostró en 1998 que una máquina dedicada podía romper DES en menos de 3 días. Hoy en día, hardware de propósito general con suficiente paralelismo puede reducir ese tiempo drásticamente.

2.2 Diagrama de flujo del algoritmo DES

La Figura 1 muestra el cifrado de un bloque de 64 bits (columna izquierda), la generación de subllaves (columna derecha) y el detalle de la función de ronda F (parte inferior). Para descifrar se usa exactamente el mismo diagrama aplicando las subllaves en el orden $K_{16}, K_{15}, \ldots, K_1$.

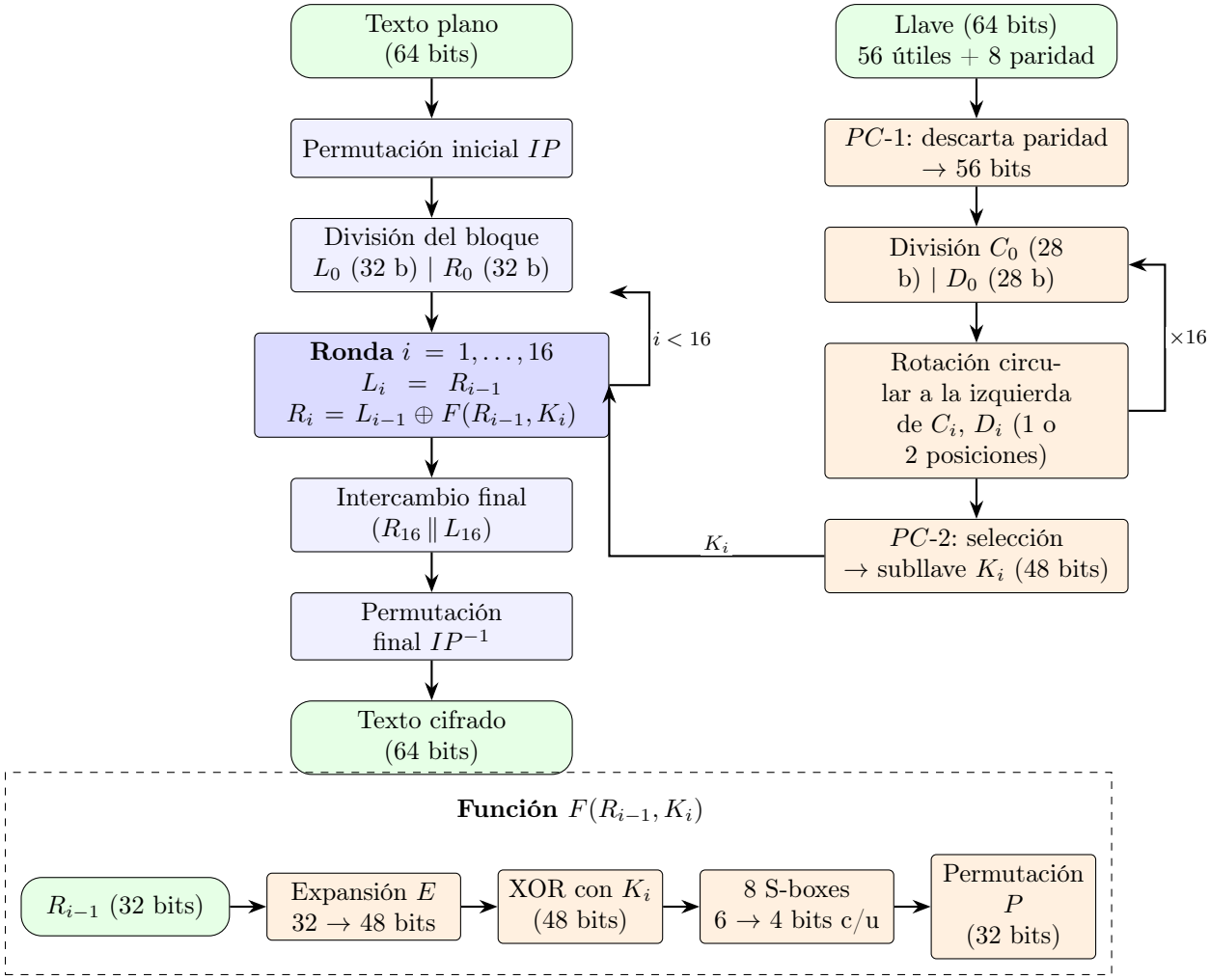


Figura 1: Diagrama de flujo de DES: cifrado de un bloque, generación de subllaves y función de ronda F.

2.3 Resultados de la versión secuencial

La versión secuencial (`src/bruteforce/secuencial.c`) recorre linealmente el rango $[0, \texttt{max_key}]$ probando cada llave con `try_key`, y sirve como línea base para calcular el speedup de todas las versiones paralelas.

Llave buscada	**Iteraciones**	**Tiempo (s)**	**Tasa (llaves/s)**
123,456 (base, Parte B.2)	123,457	0.039	$3{,}17 \times 10^6$
67,108,865 ($N/2+1$, fácil)	67,108,866	20.890	$3{,}21 \times 10^6$
83,886,080 ($N/2+N/8$, media)	83,886,081	26.097	$3{,}21 \times 10^6$
29,498,402 ($N/7+N/13+1$, difícil)	29,498,403	9.197	$3{,}21 \times 10^6$

Cuadro 1: Versión secuencial, $N = 2^{27}$ (`scripts/benchmark.sh`, `resultados_benchmark.csv`). La secuencial prueba las llaves en orden y se detiene al encontrar la correcta, así que hace exactamente *llave*+1 iteraciones, y su tiempo es *iteraciones*/*tasa* con una tasa constante de $\approx 3{,}21 \times 10^6$ llaves/s. Por eso la tabla es lineal: sirve como línea base (T_{seq}) y para medir la tasa, no como análisis. Los nombres fácil/media/difícil describen qué tan favorable es la llave para naive con 4 procesos (Sección 3.4), no para la secuencial, para la cual la llave más grande es la más lenta.

El espacio de llaves real de DES, $N = 2^{56}$, es intratable en la práctica para esta comparación. A la tasa de verificación medida en esta máquina ($\approx 3{,}2 \times 10^6$ llaves/segundo por núcleo), recorrer 2^{56} llaves secuencialmente tomaría aproximadamente $2{,}25 \times 10^9$ segundos, unos 71 años. Por eso se sustituyó $N = 2^{56}$ por un espacio efectivo reducido $N_{\text{ef}} = 2^{27} \approx 1{,}34 \times 10^8$ para los casos fácil/media/difícil de la Parte B.5, siguiendo la misma lógica de fracciones que propone el enunciado. Esto preserva el fenómeno que se quiere observar, la posición relativa dentro del segmento, con tiempos de ejecución de segundos en lugar de décadas.

2.4 Evidencia de correcto funcionamiento del wrapper DES

Se verificó el correcto funcionamiento del wrapper de cifrado/descifrado ejecutando el ciclo completo de cifrar y luego descifrar con la misma llave, confirmando que se recupera exactamente el texto original:

```
bin/cifrar data/texto.txt data/texto_cifrado.bin 42
bin/descifrar data/texto_cifrado.bin 42
# Salida esperada: "Esta es una prueba de proyecto 2"
```

Ver Figura 4 (Sección 3.1).

Las funciones `des_encrypt` y `des_decrypt` de `src/common/des.c` son un wrapper sobre la API de DES de OpenSSL (`DES_ecb_encrypt`, OpenSSL Project, 2024). Se usa en lugar de la cabecera histórica `rpc/des_crypt.h`, que ya no está disponible en distribuciones modernas de Linux. La llave, representada como un entero `long`, se empaqueta en un bloque de 8 bytes a razón de 7 bits útiles por byte, con el bit 0 siempre en 0, antes de construir el *key schedule* con `DES_set_key_unchecked`. Esta función, a diferencia de `DES_set_key`, no valida ni fuerza los bits de paridad.

Se empaquetan sólo 7 bits por byte porque DES ignora el bit menos significativo de cada byte (paridad): con los 8 bits crudos, llaves que difieren sólo en esos bits darían el mismo *key schedule*.

Así cada valor de `long` en $[0, 2^{56} - 1]$ corresponde a una llave DES distinta, y el espacio efectivo recorrido es de 2^{56} llaves distinguibles, como indica el enunciado.

2.5 Explicación de rutinas clave

Las siguientes rutinas de `src/common/` son compartidas por los 8 binarios del proyecto (cifrado, descifrado y las 6 versiones de bruteforce). Su detalle de entradas/salidas se amplía en el Anexo 1 (Sección 5).

Funcionamiento de cada rutina

`des_encrypt` / `des_decrypt` **(equivalentes a** `encrypt`/`decrypt` **de** `rpc/des_crypt.h`**).** Reciben la llave como entero `long` y un buffer cuya longitud es múltiplo de 8 B. Primero `long_to_des_key` empaqueta la llave en un bloque de 8 bytes (7 bits útiles por byte, bit de paridad en 0). Luego `DES_set_key_unchecked` calcula el *key schedule* (las 16 subllaves de la Figura 1) una sola vez, y `DES_ecb_encrypt` procesa cada bloque de 8 B de forma independiente (modo ECB) modificando el buffer *in-place*. El descifrado es idéntico pero con el flag `DES_DECRYPT`, que aplica las subllaves en orden inverso.

`try_key` **(equivalente a** `tryKey`**).** Decide si una llave candidata es la correcta. Descifra una *copia* del texto cifrado y verifica si el resultado contiene la palabra clave (Figura 2). Es la rutina que se ejecuta N veces en la búsqueda, por lo que domina el costo total: a la tasa medida son $\approx 3{,}2 \times 10^6$ llamadas por segundo por núcleo.

`memcpy`**.** Copia `len` bytes de `ciph` a un buffer temporal `temp`. Es necesaria porque `des_decrypt` trabaja *in-place*: sin la copia, el primer intento destruiría el texto cifrado y los intentos siguientes descifrarían basura. Se reserva `len+1` bytes para poder escribir un `'\0'` al final.

`strstr`**.** Recorre `temp` buscando la primera aparición de `keyword` y devuelve un puntero a ella o `NULL`. Como opera sobre cadenas terminadas en `'\0'`, el terminador explícito `temp[len]=0` es indispensable. Con una llave incorrecta el descifrado produce bytes pseudo-aleatorios, y la probabilidad de que contengan por azar la frase de 16 caracteres es $\approx 2^{-128}$ por posición, es decir, despreciable.

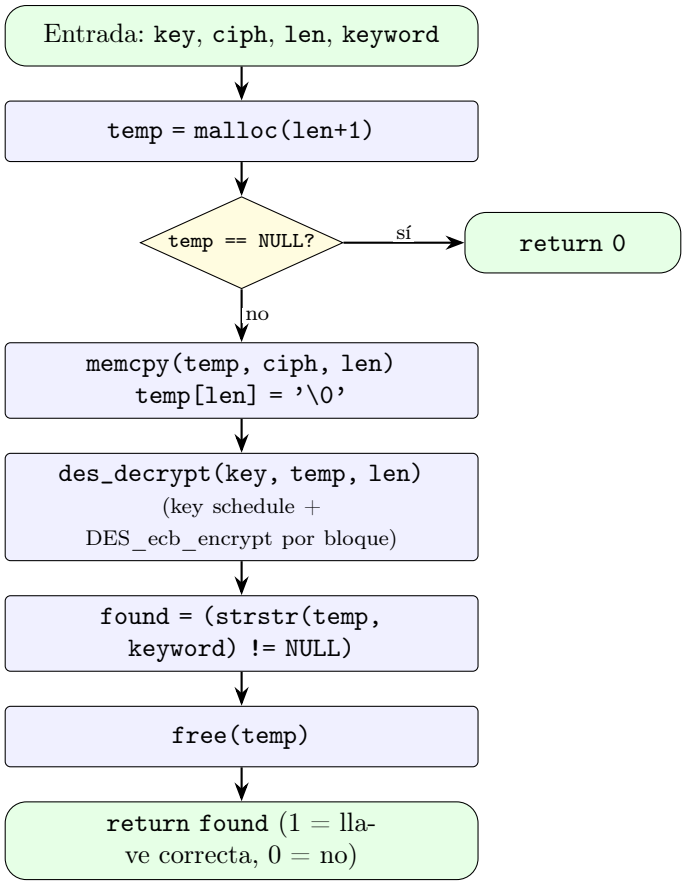


Figura 2: Diagrama de flujo de `try_key`.

2.6 Primitivas MPI utilizadas

El coordinador de "llave encontrada" se implementó con las siguientes primitivas de Open MPI, siguiendo el modelo de paso de mensajes descrito por Gropp, Lusk y Skjellum (2014).

Primitiva	**Uso en el proyecto**
`MPI_Irecv`	Postea recepción no bloqueante del aviso de "llave encontrada", permitiendo seguir buscando mientras se espera
`MPI_Test`	Sondea periódicamente si llegó el aviso, sin bloquear la búsqueda local
`MPI_Send`	Envía la llave encontrada, uno a uno, al resto de procesos para que detengan su búsqueda
`MPI_Cancel`	Cancela la recepción pendiente (`MPI_Irecv`) en el proceso que encontró la llave localmente, antes de cerrarla con `MPI_Wait`
`MPI_Wait`	Espera (bloqueante) a que se complete la recepción pendiente antes de finalizar
`MPI_Reduce`	Recolecta el tiempo máximo de ejecución y la llave global encontrada en el proceso 0
`MPI_Init_thread`	Inicializa MPI con soporte `MPI_THREAD_FUNNELED`, requerido por la versión híbrida con OpenMP

Cuadro 2: Primitivas de Open MPI utilizadas a través de los distintos acercamientos.

Flujo de comunicación de `MPI_Irecv`, `MPI_Send` y `MPI_Wait`

`MPI_Irecv` no bloquea: se postea antes de buscar y devuelve un manejador. `MPI_Test` consulta sin bloquear (cada 100 000 iteraciones). `MPI_Send` lo ejecuta quien encuentra la llave, una vez por cada otro proceso. `MPI_Wait` bloquea hasta completar la solicitud pendiente (Figura 3).

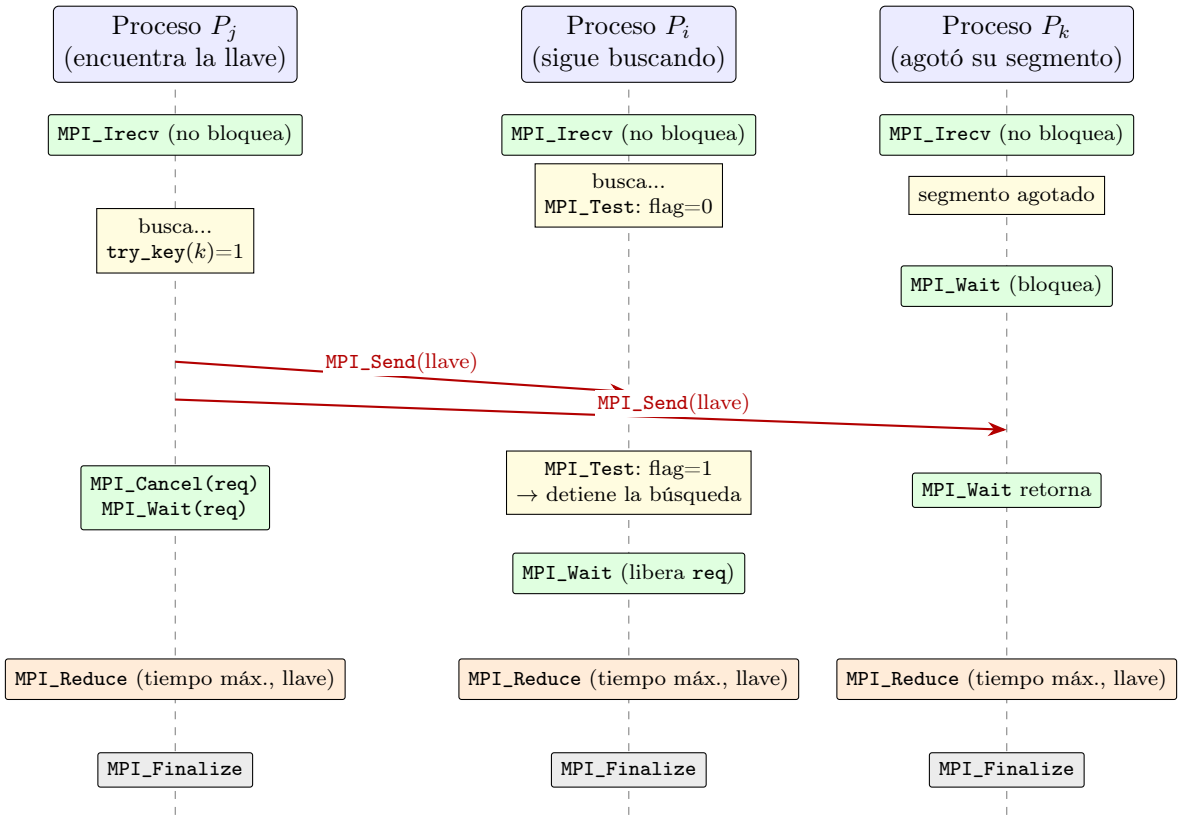


Figura 3: Flujo de comunicación del aviso de "llave encontrada" (acercamientos naive, striding, shuffled e híbrido). El maestro del acercamiento master-worker usa `MPI_Send`/`MPI_Recv` bloqueantes con las etiquetas `TAG_WORK`, `TAG_RESULT` y `TAG_STOP`.

3 Parte B — Análisis y acercamientos paralelos

3.1 Cifrado y descifrado de un texto sencillo

```
bin/cifrar data/texto.txt data/texto_cifrado.bin 42
# -> genera data/texto_cifrado.bin (texto cifrado con DES, llave=42)

bin/descifrar data/texto_cifrado.bin 42
# -> imprime: "Esta es una prueba de proyecto 2"
```

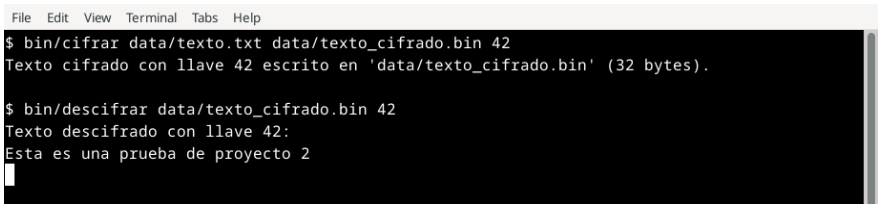


Figura 4: Ejecución real de `cifrar` y `descifrar` con llave 42.

3.2 Pruebas con `-np 4` sobre el texto de referencia

Se utilizó el texto de referencia del enunciado, *"Esta es una prueba de proyecto 2"*, buscando la palabra clave *"es una prueba de"* con el acercamiento naive y 4 procesos MPI.

a) Llave 123456

```
bin/cifrar data/texto.txt data/texto_cifrado.bin 123456
mpirun -np 4 bin/naive_mpi data/texto_cifrado.bin "es una prueba de" 72057594037927935
```

Resultado medido: llave encontrada en **0.094 s** (tiempo paralelo interno, `MPI_Wtime`). La llave cae en el primer segmento del espacio $[0, 2^{56} - 1]$, a unas 123 456 iteraciones del inicio del proceso 0.

b) Llave 18014398509481983 ($2^{56}/4$, borde entre procesos 0 y 1)

```
bin/cifrar data/texto.txt data/texto_cifrado.bin 18014398509481983
mpirun -np 4 bin/naive_mpi data/texto_cifrado.bin "es una prueba de" 72057594037927935
```

Este caso cae al final del segmento del proceso 0. Es el peor caso posible para esa llave dentro de su segmento, así que el tiempo de ejecución se aproxima al de la versión secuencial completa sobre un cuarto del espacio.

Resultado medido: no termina. Se interrumpió a los 20 s con `timeout`, como anticipa el enunciado. El proceso 0 debe recorrer $2^{54} \approx 1{,}8 \times 10^{16}$ llaves, que a $3{,}2 \times 10^6$ llaves/s son unos 180 años.

c) Llave 18014398509481984 ($2^{56}/4 + 1$)

```
bin/cifrar data/texto.txt data/texto_cifrado.bin 18014398509481984
mpirun -np 4 bin/naive_mpi data/texto_cifrado.bin "es una prueba de" 72057594037927935
```

Al estar justo al inicio del segmento del proceso 1, este caso produce un speedup artificialmente alto. El proceso 1 encuentra la llave casi de inmediato.

Resultado medido: llave encontrada en **0.031 s**. Un algoritmo secuencial habría necesitado $2^{56}/4 + 1$ iteraciones, así que el speedup aparente es astronómico (y engañoso), tal como advierte el enunciado.

Nota: en (a), (b) y (c) `max_key` es $2^{56} - 1$ (72057594037927935) para que el rango se divida en cuatro bloques de 2^{54} llaves.

d) Reflexión sobre el comportamiento

El tiempo observado en el acercamiento naive depende casi exclusivamente de la posición relativa de la llave solución dentro de su segmento contiguo, no del tamaño absoluto de `max_key`. Si la llave cae cerca del inicio del segmento de algún proceso, ese proceso la encuentra rápido y notifica al resto mediante `MPI_Send`. Si cae cerca del final del segmento más largo, el tiempo total se aproxima al de la búsqueda secuencial sobre ese mismo segmento. El mismo `max_key` puede dar speedups radicalmente distintos según en qué mitad de qué segmento caiga la solución real. Esta alta varianza es la motivación central detrás de los tres acercamientos alternativos descritos a continuación. Todos buscan reducir la dependencia entre la posición de la llave y el tiempo de descubrimiento.

El caso (b) no se puede ejecutar completo: el trabajo total sigue siendo $O(N)$, y a $3{,}2 \times 10^6$ llaves/s son años aun con 4 procesos. Por eso las comparaciones fácil/media/difícil (Sección 3.4) usan un espacio reducido $N_{\text{ef}} = 2^{27}$.

3.3 Acercamientos alternativos

Referencia: acercamiento naive

`src/bruteforce/naive_mpi.c` divide $[0, \texttt{max_key}]$ en np segmentos contiguos de igual tamaño. La Figura 5 compara cómo cada acercamiento reparte las llaves entre 4 procesos.

```
Para cada proceso (rank r de np):
  tam   <- (max_key + 1) / np
  inicio <- r * tam ;  fin <- (r == np-1) ? max_key : inicio + tam - 1
  MPI_Irecv(aviso)                        # aviso de "llave encontrada"
  para key desde inicio hasta fin:
    si try_key(key): MPI_Send(key) a los demas; MPI_Cancel(aviso); terminar
    si (key - inicio) % CHECK_INTERVAL == 0 y MPI_Test(aviso): terminar
  si se agoto el segmento: MPI_Wait(aviso)   # espera a quien la tenga
  MPI_Reduce(tiempo maximo, llave) hacia rank 0
```

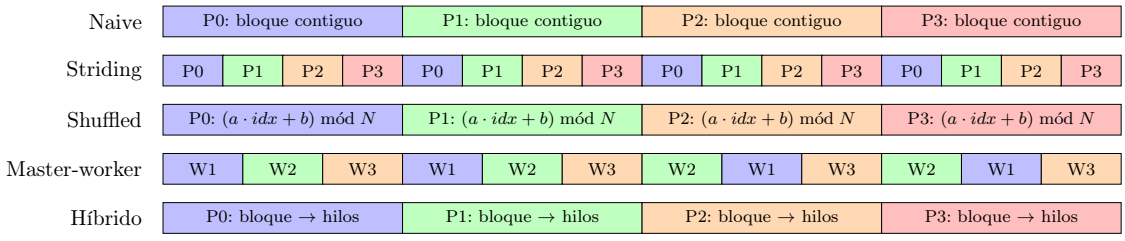


Figura 5: Reparto del espacio de llaves $[0, N-1]$ (de izquierda a derecha) entre 4 procesos. Cada color es un proceso (en master-worker, un trabajador; el rank 0 solo coordina). En striding los colores se alternan llave a llave; en master-worker, bloque a bloque y bajo demanda; en shuffled cada bloque contiguo de índices se dispersa por todo el espacio.

3.3.1. Acercamiento 1 — Master-Worker dinámico

Archivo: `src/bruteforce/master_worker_mpi.c`

El proceso de rank 0 actúa exclusivamente como maestro y no participa en la búsqueda. Distribuye bloques pequeños de tamaño `chunk_size` (por defecto, 50,000 llaves) a los procesos trabajadores (`rank` ≥ 1) bajo demanda. Cuando un trabajador termina de recorrer su bloque sin encontrar la llave, pide el siguiente al maestro. Esta estrategia de *pull* elimina la varianza observada en el acercamiento naive, porque ningún proceso queda permanentemente atado a un segmento desfavorablemente largo. En cuanto un proceso libera capacidad, se le asigna más trabajo.

```
Maestro (rank=0):              Trabajador (rank>=1):
  reparte bloque inicial  -->    recibe [inicio, fin]
  loop:                          loop:
    recibe resultado       <--     busca en [inicio, fin]
    si llave hallada:              envia resultado
      envia TAG_STOP a todos -->   si TAG_STOP: termina
    si hay mas trabajo:            si TAG_WORK: repite
      envia TAG_WORK         -->
```

El costo de esta estrategia es el overhead de comunicación. Cada bloque consumido implica una ronda adicional de mensajes maestro-trabajador, lo que puede volverse significativo si `chunk_size` es muy pequeño en relación con la latencia de red.

```
mpirun -np 4 bin/master_worker_mpi data/texto_cifrado.bin "es una prueba de" <max_key> [
    chunk_size]
```

Las ejecuciones reales de los 5 acercamientos (incluyendo este) sobre el mismo caso de prueba se muestran juntas en la Figura 6, al final de esta subsección.

3.3.2. Acercamiento 2 — Intercalado / Striding

Archivo: `src/bruteforce/striding_mpi.c`

En lugar de un bloque contiguo, cada proceso de rank r recorre las llaves $r, r+np, r+2\cdot np, \ldots$, donde np es el número total de procesos. De esta forma, todos los procesos exploran simultáneamente la misma "profundidad relativa" del espacio de búsqueda en todo momento. Esto elimina por completo la correlación entre la posición absoluta de la llave solución y qué proceso tiene ventaja. Sin importar dónde caiga la llave, en promedio todos los procesos llevan un avance similar cuando se encuentra.

```
Para cada proceso (rank r de np):
  key <- r
  mientras key <= max_key:
    si try_key(key): avisar a los demas y terminar
    si (key - r) % CHECK_INTERVAL == 0: revisar aviso (MPI_Test)
    key <- key + np        # salto de tamano np: entrelazado
```

```
mpirun -np 4 bin/striding_mpi data/texto_cifrado.bin "es una prueba de" <max_key>
```

Ver Figura 6.

3.3.3. Acercamiento 3 — Partición contigua con shuffle pseudo-aleatorio

Archivo: `src/bruteforce/shuffled_mpi.c`

Este acercamiento conserva la misma partición en bloques contiguos que el naive (y, por tanto, la misma simplicidad de asignación estática de trabajo), pero en lugar de recorrer cada bloque en

orden ascendente, cada índice lógico `idx` dentro del bloque se mapea a la llave real mediante una función de congruencia lineal:

$$\texttt{key}(\texttt{idx}) = (a \cdot \texttt{idx} + b) \bmod N$$

donde $N = \texttt{max_key} + 1$ y a se elige coprimo con N. Esta construcción garantiza una biyección sobre $[0, N-1]$: cada llave del espacio se visita exactamente una vez en total. Pero el *orden* en que cada proceso las visita queda "barajado" de forma determinista y reproducible. No es una permutación aleatoria real, sino pseudo-aleatoria vía aritmética modular. El efecto práctico es similar al striding, rompe la correlación entre posición y tiempo de hallazgo, pero sin necesitar que los procesos avancen en sincronía por todo el espacio.

```
N <- max_key + 1
[idx_start, idx_end] <- segmento contiguo del proceso (igual que naive)
para idx desde idx_start hasta idx_end:
  key <- (a * idx + b) mod N      # biyeccion pseudo-aleatoria
  si try_key(key): avisar a los demas y terminar
  cada CHECK_INTERVAL: revisar aviso (MPI_Test)
```

```
mpirun -np 4 bin/shuffled_mpi data/texto_cifrado.bin "es una prueba de" <max_key>
```

Ver Figura 6.

3.3.4. Acercamiento 4 — Híbrido MPI + OpenMP

Archivo: `src/bruteforce/hybrid_mpi_omp.c`

Esta versión combina memoria distribuida (MPI) con memoria compartida dentro de cada nodo (OpenMP, OpenMP ARB, 2021). Añade un segundo nivel de paralelismo dentro de cada proceso MPI: el bloque contiguo asignado a cada proceso se subdivide entre los hilos de OpenMP disponibles, cada uno recorriendo su sub-bloque con un stride igual al número de hilos. Es el mismo principio de striding, aplicado ahora a nivel de hilos en memoria compartida en vez de a nivel de procesos en memoria distribuida. MPI se inicializa con `MPI_Init_thread` solicitando el nivel `MPI_THREAD_FUNNELED`, ya que únicamente el hilo maestro (`omp` thread 0) de cada proceso realiza llamadas a funciones MPI. El resto de los hilos se limitan a ejecutar `try_key` en su porción del espacio de búsqueda.

El objetivo de este acercamiento es doble. Primero, reducir el número de procesos MPI necesarios para saturar los núcleos de una máquina, disminuyendo así el overhead de paso de mensajes. Segundo, aprovechar la memoria compartida dentro de cada nodo para comunicar el estado de "llave encontrada" entre hilos de forma más económica que mediante mensajes MPI.

```
Nivel 1 (por proceso MPI, rank r de np):
  [block_start, block_end] <- segmento contiguo (igual que naive)
  encontrado_local <- bandera compartida entre hilos (false)

  Nivel 2 (por hilo OpenMP, id t de nthreads), en paralelo:
    key <- block_start + t
    mientras key <= block_end y no encontrado_local:
      si try_key(key): encontrado_local <- true; guardar key; salir
      si t == 0 y toca revisar: sondear aviso MPI (MPI_Test)
      key <- key + nthreads          # entrelazado entre hilos
```

```
  (fin de la region paralela, los hilos se unen)
  si encontrado_local (hallado por este proceso):
    avisar a los demas procesos (MPI_Send)
  si no: esperar aviso de otro proceso (MPI_Wait)
```

```
mpirun -np 4 bin/hybrid_mpi_omp data/texto_cifrado.bin "es una prueba de" <max_key> [
    nthreads]
```

Ver Figura 6.

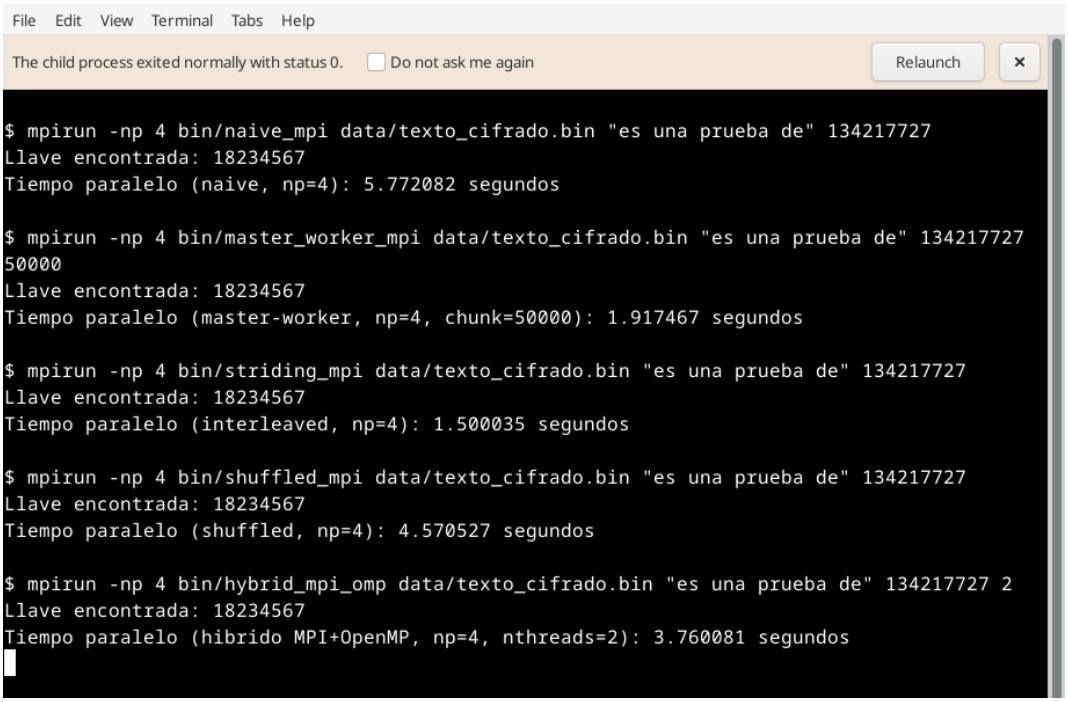


Figura 6: Ejecución real de los 5 acercamientos paralelos sobre el mismo caso de prueba (llave objetivo = 18,234,567, `max_key` = 134,217,727, $np = 4$). Los 5 encuentran exactamente la llave correcta de forma consistente.

3.4 Comparativa de tiempos (fácil / media / difícil)

Siguiendo la metodología del enunciado, se definieron tres llaves (fácil, media y difícil) con las mismas fracciones propuestas pero aplicadas sobre un espacio efectivo reducido $N_{\text{ef}} = 2^{27}$ (ver Sección 2.3). **El espacio de búsqueda es el mismo para los tres casos** (`max_key` $= 2^{27} - 1$), de modo que lo único que cambia es la posición de la llave, tal como plantea el enunciado. Se ejecutó cada uno de los seis programas (secuencial + 5 paralelos) con $np = 4$ y, para el híbrido, 2 hilos por proceso (8 hilos en total). Los tiempos son los que imprime cada programa (`MPI_Wtime` o reloj interno), es decir, el tiempo de búsqueda sin el arranque de `mpirun`. El tiempo de pared del comando completo se conserva en la columna `tiempo_wall_segundos` del CSV; difiere en $\approx 1{,}5\,\text{s}$ por el lanzamiento de MPI. Cada punto es una sola corrida (no hay repeticiones). Los resultados completos están en el Anexo 6 y en `resultados_benchmark.csv`.

Con $np = 4$ y bloques de $2^{27}/4$ llaves, la llave fácil es la primera del bloque del proceso 2, la media está a la mitad de ese bloque y la difícil ($\approx 0{,}22N$) al 88 % del bloque del proceso 0.

Caso	Secuencial	Naive	Master-W.	Striding	Shuffled	Híbrido
Base (123 456)	0.039	0.156	0.016	0.063	7.903	0.152
Fácil	20.890	0.094	6.934	5.270	0.878	0.191
Media	26.097	5.299	8.640	6.594	3.286	3.483
Difícil	9.197	9.244	3.078	2.364	8.761	6.015

Cuadro 3: Tiempo de búsqueda (s) por caso y acercamiento, $np = 4$, espacio fijo 2^{27} (`hybrid_mpi_omp` con 2 hilos por proceso). Fuente: `resultados_benchmark.csv`.

Acercamiento	Speedup (fácil / media / difícil)	Eficiencia (fácil / media / difícil)	Divisor de eficiencia
Naive	222.9 / 4.92 / 0.99	55.7 / 1.23 / 0.25	$np = 4$
Master-worker	3.01 / 3.02 / 2.99	0.75 / 0.76 / 0.75	$np = 4$
Striding	3.96 / 3.96 / 3.89	0.99 / 0.99 / 0.97	$np = 4$
Shuffled	23.8 / 7.94 / 1.05	5.9 / 1.99 / 0.26	$np = 4$
Híbrido (2 hilos)	109.2 / 7.49 / 1.53	13.6 / 0.94 / 0.19	$np \times$hilos $= 8$

Cuadro 4: Speedup (T_{seq}/T_{par}) y eficiencia (speedup dividido entre las unidades de cómputo) respecto a la versión secuencial (calculado con `scripts/speedup.py`).

Modelo analítico del tiempo paralelo

Los resultados se pueden predecir sin ejecutar nada. Sea k la llave, N el tamaño del espacio, p el número de procesos, $B = N/p$ el tamaño del bloque y $r \approx 3{,}21 \times 10^6$ llaves/s la tasa medida de un núcleo (Tabla 1). El tiempo paralelo es el del proceso que encuentra la llave, es decir, las iteraciones que le toca hacer hasta llegar a ella, divididas entre r:

Acercamiento	Iteraciones hasta hallar k	Speedup
Secuencial	$k + 1$	1
Naive	$k - \lfloor k/B \rfloor B + 1$ (posición dentro de su bloque)	$\dfrac{k+1}{k - \lfloor k/B \rfloor B + 1}$, entre 1 y $\approx k$
Striding	$\approx k/p$ (todos avanzan al mismo ritmo)	$\approx p$
Master-worker	$\approx k/(p-1)$ (el maestro no busca)	$\approx p - 1$
Shuffled	$idx - \lfloor idx/B \rfloor B + 1$, con $idx = a^{-1}(k-b)$ mód N	depende de idx

La Tabla 5 compara este modelo con las mediciones (las llaves de naive y shuffled se calcularon con los mismos a y b que usa el programa).

	Naive		Striding		Master-W.		Shuffled	
Caso	pred.	med.	pred.	med.	pred.	med.	pred.	med.
Base	0.038	0.156	0.010	0.063	0.013	0.016	7.81	7.90
Fácil	≈ 0	0.094	5.23	5.27	6.97	6.93	0.80	0.88
Media	5.23	5.30	6.53	6.59	8.71	8.64	3.19	3.29
Difícil	9.19	9.24	2.30	2.36	3.06	3.08	8.64	8.76

Cuadro 5: Tiempo paralelo predicho por el modelo vs. medido (s), $np = 4$, $N = 2^{27}$.

Las predicciones coinciden con lo medido (diferencias de ≈1–3 % cuando el tiempo es de varios segundos). Los desvíos en las filas de tiempos muy pequeños (naive 0.038 vs. 0.156) son un costo fijo de ≈ 0,1 s de aviso y cierre entre procesos (`MPI_Send`/`MPI_Wait`), que el modelo no incluye. Esto significa que los resultados no son casualidad ni ruido: están determinados por la posición de la llave. El híbrido no se incluye porque no sigue el modelo: con 2 hilos esperaríamos la mitad del tiempo de naive (2.6 s en media, 4.6 s en difícil) y se midió 3.5 s y 6.0 s; no se investigó la causa.

Qué implica para el tiempo paralelo esperado. Si la llave es uniforme en $[0, N)$, la posición dentro del bloque de naive es uniforme en $[0, B)$, así que su tiempo esperado es $B/2 = N/(2p)$ iteraciones. El de striding es $E[k]/p = N/(2p)$. **Los dos tienen el mismo tiempo esperado y el mismo peor caso (N/p).** Lo que cambia es la dispersión del speedup: naive depende de la posición (de ≈ 1 a $\approx k$), mientras que striding da $\approx p$ siempre. Master-worker tiene un tiempo esperado peor en un factor $p/(p-1)$ en una sola máquina homogénea, y su ventaja (balancear nodos de distinta velocidad) no se midió. Shuffled convierte el speedup de naive en una variable pseudo-aleatoria sin mejorar su valor medio.

Observaciones sobre la Tabla 4:

- **Naive** va de 222.9 a 0.99 según dónde caiga la llave, exactamente el fenómeno del enunciado. Su eficiencia > 1 es suerte posicional, no ganancia paralela.
- **Striding** es el único consistente: $\approx np$ (3.96, 3.96, 3.89) sin importar la llave. A cambio nunca tiene el golpe de suerte de naive (fácil: 5.27 s vs. 0.094 s).
- **Master-worker** rinde $np - 1 \approx 3{,}0$: el rank 0 sólo coordina. El costo de los mensajes (un bloque de 50 000 llaves son ≈ 16 ms de cómputo) es pequeño frente a esa pérdida.
- **Shuffled** no elimina la varianza, la aleatoriza: 23.8, 7.94 y 1.05; con la llave base tarda 7.9 s, unas 200 veces más que la secuencial.
- **Híbrido** conserva los bloques contiguos de naive (109, 7.49, 1.53). Es 1,5× más rápido que naive en media y difícil, y más lento en fácil (0.19 vs. 0.094 s, ambos de fracción de segundo). Con 2 hilos se esperaría 2×; no se investigó la causa.

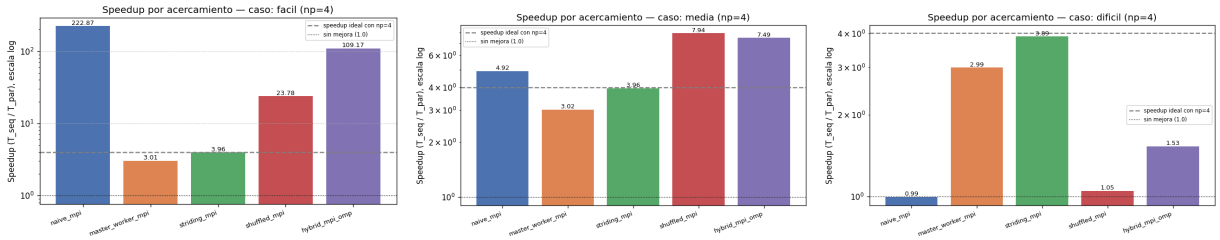


Figura 7: Speedup por acercamiento (escala log), $np = 4$: caso fácil, media y difícil.

4 Discusión, conclusiones y recomendaciones

Retos encontrados y lecciones aprendidas

El reto técnico más significativo no estuvo en la paralelización en sí, sino en la representación de la llave: DES ignora el bit de paridad de cada byte, así que los 56 bits de la llave numérica se empaquetan a razón de 7 por byte para que cada valor corresponda a una llave DES distinta (Sección 2.4). En un bruteforce conviene verificar esa correspondencia uno a uno antes de medir tiempos.

El segundo reto fue de escala. El enunciado define los casos fácil/media/difícil como fracciones de 2^{56}, pero a la tasa de verificación medida en esta máquina esas fracciones representan años de cómputo, incluso paralelizadas. Esto llevó a sustituir 2^{56} por un espacio efectivo reducido ($N_{\text{ef}} = 2^{27}$) que preserva el fenómeno de interés, la posición relativa dentro del segmento, con tiempos de ejecución manejables (Sección 2.3). Es una decisión de diseño experimental, no un atajo que invalide el análisis.

El tercer reto fue el diseño de las pruebas: el espacio de búsqueda debe ser el mismo en todos los casos y variar sólo la posición de la llave, y el tiempo debe medirse dentro del programa (el arranque de `mpirun` añade $\approx 1{,}5\,\text{s}$ al tiempo de pared).

Por último, se aprendió que con un texto de prueba y palabra clave tan cortos, un descifrado con la llave incorrecta puede, por azar, contener la palabra clave buscada. Eso sería un falso positivo. No se observó en la práctica en las corridas finales reportadas en este informe, pero es una limitación real del criterio de validación por *substring*. Vale la pena mencionarla para cualquier extensión futura del proyecto, por ejemplo usando una frase clave más larga o verificando un segundo criterio de coincidencia.

- **Varianza del naive: teoría y evidencia.** El caso analítico de la Sección 3.2 lo muestra en el extremo: con $np = 4$ y 2^{56} llaves, la misma `max_key` da un speedup de ≈ 1 si la llave está al final del bloque del proceso 0 y de $\approx 4{,}5 \times 10^{15}$ si está al inicio del bloque del proceso 1. Los casos medidos a escala 2^{27} lo confirman: naive va de 222.9 a 0.99 (coeficiente de variación del speedup entre los tres casos: 1.36). Striding, en cambio, da 3.96, 3.96 y 3.89 (CV 0.01). Son sólo tres llaves por acercamiento y una corrida por punto, por lo que el CV es ilustrativo, no una prueba estadística; una validación rigurosa requeriría muchas llaves aleatorias.
- **Compromiso entre consistencia y suerte (Sección 3.4).** Naive y shuffled (CV 0.87) pueden ganar mucho o nada según dónde esté la llave; striding y master-worker rinden siempre lo mismo ($\approx np$ y $\approx np-1$). El modelo analítico muestra que naive y striding tienen el mismo tiempo paralelo esperado ($N/2p$ iteraciones) y el mismo peor caso; la mejora de striding es de *consistencia* (speedup $\approx p$ para cualquier llave), no de tiempo esperado. Si se busca predecibilidad, striding es la mejor opción medida aquí; el costo es renunciar a los casos de "suerte".
- **Overhead de comunicación.** Master-worker envía un mensaje por bloque, pero su speedup de $\approx 3{,}0$ se explica por tener un núcleo menos buscando. Con un `chunk_size` mucho menor o más latencia podría pesar más; no se midió.
- **Beneficio del híbrido.** Combinar MPI con OpenMP permite usar la memoria compartida dentro de cada nodo sin lanzar un proceso MPI por núcleo. Frente a naive con el mismo np, el híbrido con 2 hilos fue 1,5× más rápido en los casos media y difícil (Tabla 3); en el caso

fácil fue más lento (0.19 vs. 0.094 s) porque ambos terminan en una fracción de segundo. La mejora es consistente donde hay trabajo real, pero menor que el 2× esperable, y su eficiencia sobre 8 hilos es 0.94 (media) y 0.19 (difícil, donde sigue limitado por el bloque del proceso 0). El extra de OpenMP se cumple de forma parcial, y conviene decirlo así.

- **Recomendación.** Para trabajo futuro, valdría la pena extender las pruebas a un cluster MPI de 2+ máquinas físicas (ver extras del enunciado), para observar cómo se comportan estos mismos acercamientos cuando la comunicación deja de ser intra-nodo y pasa a depender de la latencia de una red local real.

En conclusión, el programa base funciona correctamente de principio a fin. Recibe el texto plano desde archivo, lo cifra con una llave arbitraria pasada por parámetro, y las seis variantes de búsqueda (secuencial y cinco paralelas) recuperan exactamente esa misma llave, sin colisiones ni falsos positivos. Todas imprimen la llave encontrada, el archivo de entrada y la palabra clave usada en cada corrida (Secciones 3.1 y 3.4).

5 Anexo 1 — Catálogo de funciones y librerías

`des_encrypt` — `src/common/des.c`

```
void des_encrypt(long key, unsigned char *buf, size_t len)
```

- **Entradas:** `key` (long, 0 a $2^{56} - 1$), `buf` (buffer a cifrar, múltiplo de 8 B), `len` (tamaño en bytes).
- **Salidas:** modifica `buf` in-place (cifrado DES en modo ECB).
- **Descripción:** convierte `key` a un bloque de 8 bytes, invoca `DES_set_key_unchecked` y `DES_ecb_encrypt` de OpenSSL sobre cada bloque de 8 bytes del buffer.

`des_decrypt` — `src/common/des.c`

```
void des_decrypt(long key, unsigned char *buf, size_t len)
```

- Idéntica a `des_encrypt`, pero invoca `DES_ecb_encrypt` con el flag `DES_DECRYPT`.

`try_key` — `src/common/des.c`

```
int try_key(long key, const unsigned char *ciph, size_t len, const char *keyword)
```

- **Entradas:** `key`, `ciph` (texto cifrado, no se modifica), `len`, `keyword`.
- **Salidas:** `int` — 1 si `keyword` aparece en el texto descifrado, 0 en caso contrario.
- **Descripción:** copia `ciph` en un buffer temporal, llama a `des_decrypt` sobre la copia, y busca `keyword` con `strstr`. Trabajar sobre una copia evita mutar el texto cifrado original entre intentos.

`load_and_pad_file` — `src/common/common.c`

```
unsigned char *load_and_pad_file(const char *path, size_t *out_len)
```

- **Entradas:** `path` (ruta al archivo), `out_len` (puntero de salida).
- **Salidas:** puntero a buffer (el caller debe liberarlo con `free`), longitud resultante en `*out_len`.
- **Descripción:** abre el archivo, calcula el padding necesario al múltiplo de 8 más cercano, reserva memoria con `calloc`, rellena el padding con espacios y lee el contenido del archivo.

`die` — `src/common/common.c`

```
void die(const char *msg)
```

- **Entradas:** `msg` (cadena de error).
- **Salidas:** no retorna; invoca `exit(ERR_EXIT)`.
- **Descripción:** imprime `[ERROR] msg` en `stderr` y termina el proceso con el código de error del proyecto.

`long_to_des_key` — `src/common/des.c` **(privada)**

```
static void long_to_des_key(long lkey, DES_cblock *des_key)
```

- **Entradas:** `lkey` (long, llave de 56 bits), `des_key` (puntero a bloque de 8 bytes a llenar).
- **Salidas:** escribe en `*des_key` 8 bytes; cada uno contiene 7 bits de `lkey` desplazados una posición, con el bit de paridad en 0.
- **Descripción:** garantiza que a cada valor de `lkey` le corresponda un *key schedule* distinto (Sección 2.4).

`next_block_end` — `src/bruteforce/master_worker_mpi.c` **(privada)**

```
static long next_block_end(long next_key, long chunk_size, long max_key)
```

- **Entradas:** `next_key` (primera llave del bloque), `chunk_size` (tamaño del bloque), `max_key` (límite del espacio).
- **Salidas:** `long` — última llave del bloque, `min(next_key + chunk_size - 1, max_key)`.
- **Descripción:** evita que el último bloque exceda el espacio de búsqueda.

`gcd`, `mulmod`, `find_coprime_multiplier` — `src/bruteforce/shuffled_mpi.c` **(privadas)**

```
static long gcd(long a, long b)  static long mulmod(long a, long b, long n)  static long find_coprime_mu
n)
```

- **Entradas:** `a`, `b` (enteros no negativos), `n` (tamaño del espacio, $N = \texttt{max_key} + 1$).
- **Salidas:** `gcd` devuelve el máximo común divisor (algoritmo de Euclides); `mulmod` devuelve $(a \cdot b)$ mód n; `find_coprime_multiplier` devuelve el menor $a \geq$ semilla con $\gcd(a, n) = 1$.
- **Descripción:** `mulmod` usa `__int128` como acumulador porque $a \cdot b$ puede superar los 64 bits cuando $n \approx 2^{56}$. Un a coprimo con n es condición necesaria y suficiente para que $key(idx) = (a \cdot idx + b)$ mód n sea una biyección en $[0, n-1]$.

Programas ejecutables (`main`)

Todos validan el número de argumentos y que `max_key` ≥ 0, y terminan con `ERR_EXIT` (=1) si no se cumplen. Los archivos inexistentes o vacíos terminan vía `die()` dentro de `load_and_pad_file`.

Programa	Entradas (`argv`)	Salidas (stdout)
`cifrar`	`archivo_entrada` (char*), `archivo_salida` (char*), `llave` (long)	Escribe el cifrado en `archivo_salida`; imprime llave y bytes escritos
`descifrar`	`archivo_cifrado` (char*), `llave` (long)	Imprime el texto descifrado
`secuencial`	`archivo_cifrado`, `palabra_clave` (char*), `max_key` (long)	Archivo y palabra usados, `Llave encontrada: k`, tiempo (s)
`naive_mpi`, `striding_mpi`, `shuffled_mpi`	igual que `secuencial`; `np` vía `mpirun -np`	igual, con el tiempo máximo entre procesos (`MPI_Reduce`)
`master_worker_mpi`	igual + `chunk_size` (long, opcional, 50 000); requiere $np \geq 2$	igual
`hybrid_mpi_omp`	igual + `nthreads` (int, opcional)	igual

Cuadro 6: Interfaz de línea de comandos de los ocho binarios.

Librerías externas

- **OpenSSL / libcrypto** (`openssl/des.h`): provee las primitivas de bajo nivel de DES usadas por `des_encrypt`/`des_decrypt` (arriba).
- **Open MPI** (`mpi.h`): provee las primitivas de paso de mensajes listadas en la Sección 2.6, usadas en las cinco versiones paralelas.
- **OpenMP** (`omp.h`, pragma `#pragma omp parallel`): provee el paralelismo de memoria compartida dentro de cada proceso en la versión híbrida.

6 Anexo 2 — Bitácora de pruebas y speedups

El benchmark completo se ejecutó con 4 procesos, 2 hilos OpenMP para el híbrido y un timeout de 180 segundos por corrida, sobre el espacio efectivo reducido $N_{\text{ef}} = 2^{27}$ fijo (Sección 2.3). La ejecución completa (4 casos $\times$ 6 programas = 24 corridas) tomó ≈ 4 minutos. Los resultados crudos están en `resultados_benchmark.csv` (repositorio del proyecto) y el resumen de speedup/eficiencia en la Tabla 4.

Las capturas de pantalla de las ejecuciones reales se presentan en las Figuras 4 y 6, y las gráficas de speedup/eficiencia generadas por `scripts/speedup.py` se presentan en las Figuras **??**–7 (Sección 3.4).

Variación del tiempo con el número de procesos

Caso "difícil" (llave 29 498 402 dentro de $[0, 2^{27} - 1]$, secuencial 9.20 s), tiempo de búsqueda interno (`MPI_Wtime`). El híbrido usa 2 hilos por proceso, por lo que con np procesos emplea $2\,np$ núcleos. Má-

quina de 20 núcleos lógicos, corridas con `-oversubscribe`. Datos crudos en `resultados_np_scaling.csv`; una sola corrida por punto.

Acercamiento	$np = 2$	$np = 4$	$np = 8$	$np = 16$
Naive	9.06	9.31	4.07	2.03
Master-worker	9.22	3.08	1.42	0.78
Striding	4.64	2.37	1.46	0.90
Shuffled	8.74	8.77	4.09	1.41
Híbrido (2 hilos)	4.69	5.88	3.65	1.96

Cuadro 7: Tiempo (s) por acercamiento y número de procesos.

Naive no mejora entre $np = 2$ y $np = 4$ (la llave sigue en el bloque del proceso 0) y baja a saltos al achicarse los bloques ($np = 8, 16$). Striding escala casi linealmente (speedups 1.98, 3.89, 6.31, 10.3; eficiencia 0.64 en $np = 16$, no se investigó la causa). Master-worker rinde $\approx np - 1$ (con $np = 2$ hay un solo trabajador). Shuffled sigue el patrón por bloques de naive con esta llave. El híbrido usa $2\,np$ hilos y, como sus bloques son contiguos, para $np \leq 4$ la llave sigue en el bloque del proceso 0.

Pasos para reproducir las pruebas:

```
export PATH="/usr/lib64/openmpi/bin:$PATH"   # si mpirun no esta en PATH
make
chmod +x scripts/benchmark.sh
./scripts/benchmark.sh 4 180 2  # np=4, timeout=3 min, 2 hilos OpenMP
python3 scripts/speedup.py      # genera tabla + graficas en results/
```

Extras

- **Cluster MPI en 2+ máquinas:** no intentado.
- **OpenMP + MPI:** `hybrid_mpi_omp.c`. Es 1,5× más rápido que naive con el mismo `np` en los casos media y difícil, y más lento en el fácil (Tabla 3).
- **3+ acercamientos adicionales:** master-worker, striding, shuffled e híbrido, todos medidos en la Tabla 3.

7 Bibliografía

1. Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7.ª ed.). Pearson. Capítulo 3: Block Ciphers and the Data Encryption Standard.

2. National Institute of Standards and Technology (NIST). (1999). *Data Encryption Standard (DES)* (FIPS PUB 46-3). U.S. Department of Commerce. https://csrc.nist.gov/publications/detail/fips/46/3/final

3. Gropp, W., Lusk, E., & Skjellum, A. (2014). *Using MPI: Portable Parallel Programming with the Message-Passing Interface* (3.ª ed.). MIT Press.

4. OpenSSL Project. (2024). *DES routines — OpenSSL 3.x documentation*. https://www.openssl.org/docs/man3.0/man3/DES_ecb_encrypt.html

5. OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface* (versión 5.2). https://www.openmp.org/specifications/